

Dokumentasi Teknis: RAG Backend & Frontend

Audrew Alessandro Malau

September 2026

Dokumentasi Teknis Proyek RAG (Retrieval Augmented Generation)

1. Ringkasan Proyek

Proyek ini adalah sistem RAG (Retrieval Augmented Generation) yang terdiri dari dua bagian:

- **Backend** (`rag-backend`): API berbasis FastAPI (Python) yang menangani ingest dokumen, penyimpanan vector, dan percakapan dengan LLM.
- **Frontend** (`rag-frontend`): Antarmuka web berbasis Next.js yang menampilkan indeks dokumen dan ruang percakapan, lengkap dengan sitasi sumber di setiap jawaban.

Tujuan utamanya: pengguna dapat mengunggah dokumen (PDF, DOCX, PPTX) maupun data tabular (Excel, CSV), lalu bertanya dalam bahasa natural, dan sistem menjawab berdasarkan isi dokumen tersebut, bukan dari pengetahuan umum model.

2. Tools dan Teknologi yang Dipakai

Komponen	Tools yang Dipakai	Alasan Pemilihan
Model routing (LLM & embedding)	LiteLLM	Lapisan abstraksi agar provider model (Gemini, OpenAI, DeepSeek, dll) bisa diganti lewat konfigurasi saja, tanpa mengubah kode.
LLM (jawab pertanyaan)	DeepSeek , diakses lewat OpenRouter	Murah dan cukup baik untuk kebutuhan tanya jawab dokumen. Diakses lewat OpenRouter, bukan API DeepSeek langsung.
Embedding (representasi vektor teks)	all-MiniLM-L6-v2 (library <code>sentence-transformers</code>), jalan lokal	DeepSeek tidak menyediakan endpoint embedding, jadi dipakai model open source yang jalan di mesin sendiri, gratis dan tanpa API key tambahan.
Orkestrasi RAG	LangChain	Menyusun alur retrieve, susun prompt, dan generate jawaban sebagai pipeline yang terstruktur dan mudah diperiksa tiap tahapnya.
Tool calling untuk data tabular	LiteLLM function calling (pola manual tool loop)	LLM diberi akses ke satu tool (<code>query_table</code>) untuk menjalankan SQL, dipakai khusus saat pertanyaan butuh perhitungan presisi dari data Excel.
Vector database	TurboVec (<code>IdMapIndex</code>)	Index vektor lokal yang dikompresi, tidak butuh server terpisah, sesuai arahan untuk memakai TurboVec.
Parsing dokumen (PDF/DOCX/PPTX)	Docling	Memecah dokumen menjadi bagian-bagian (chunk) dengan tetap menjaga struktur, termasuk ekstraksi tabel.
Ingest Excel/CSV	pandas + openpyxl , disimpan ke SQLite	Setiap sheet diubah menjadi tabel SQL asli, bukan sekadar teks yang di-embed, agar bisa dihitung secara presisi (dijelaskan lebih lanjut di bagian 4).

Metadata penyimpanan	SQLite (<code>metadata_store.py</code>)	Mencatat dokumen apa saja yang sudah di-ingest, model embedding yang dipakai, dan detail chunk-nya.
Backend framework	FastAPI	Framework Python untuk membuat REST API, cepat dan punya dokumentasi otomatis (Swagger UI).
Frontend framework	Next.js 14 (App Router) + Tailwind CSS	Framework React untuk antarmuka web, sesuai arahan awal untuk memakai Next.js.

3. Arsitektur dan Alur Kerja

3.1 Alur Ingest Dokumen (PDF, DOCX, PPTX)

1. Pengguna mengunggah file lewat frontend.
2. Backend memakai Docling untuk memecah dokumen menjadi potongan (chunk) teks, tabel, dan gambar (opsional lewat fitur VLM).
3. Setiap potongan diubah menjadi vektor (embedding) memakai model yang dikonfigurasi.
4. Vektor disimpan ke TurboVec, dikelompokkan berdasarkan model embedding yang dipakai (jika model embedding diganti, koleksi vektor yang baru otomatis terpisah, tidak tercampur dengan koleksi lama).
5. Metadata dokumen dan tiap chunk dicatat ke SQLite.

3.2 Alur Ingest Excel/CSV

Pendekatan untuk data Excel sengaja dibuat berbeda dari dokumen biasa, karena pertanyaan terhadap data tabular biasanya butuh perhitungan (total, rata-rata, filter), bukan sekadar pencarian teks yang mirip.

1. Setiap sheet dibaca memakai pandas, lalu ditulis sebagai tabel SQL asli ke dalam sebuah file SQLite (`excel_data.sqlite3`).
2. Hanya **ringkasan struktur tabel** (nama tabel, nama kolom, beberapa baris contoh) yang diubah menjadi embedding dan disimpan ke TurboVec, bukan seluruh baris data. Ini yang membuat proses ingest tetap ringan meski jumlah barisnya besar.
3. Saat pengguna bertanya, jika hasil pencarian semantik menemukan tabel yang relevan, LLM diberi akses ke tool `query_table` untuk menulis satu query SQL `SELECT` dan mendapat jawaban yang presisi, bukan tebakan dari kemiripan teks.

3.3 Alur Percakapan (Chat)

1. Pertanyaan pengguna diubah menjadi embedding, dicari kecocokannya di TurboVec.
2. Jika hasil pencarian berisi skema tabel Excel, sistem masuk ke jalur tool calling: LLM boleh memanggil `query_table` (dibatasi maksimal beberapa putaran, hanya perintah `SELECT`, ada batas waktu dan batas jumlah baris hasil) sebelum menjawab.
3. Jika tidak, sistem memakai jalur RAG biasa: konteks dari chunk yang relevan dimasukkan ke prompt, lalu LLM menjawab langsung.
4. Jawaban dikembalikan ke frontend beserta daftar sumber (chunk atau hasil query) yang dipakai, ditampilkan sebagai catatan kaki bernomor yang bisa diklik.

3.4 Keamanan Query SQL

Karena LLM yang menulis query SQL, ada lapisan pembatas di `sql_tool.py`:

- Hanya perintah `SELECT` (boleh diawali `WITH`) yang diterima.
- Kata kunci berbahaya seperti `INSERT`, `UPDATE`, `DELETE`, `DROP`, `ALTER`, `PRAGMA` ditolak.
- Hanya satu pernyataan per pemanggilan (tidak boleh ada tanda titik koma di tengah).
- Koneksi database dipaksa mode `query_only`, sebagai lapisan pengaman tambahan di level

SQLite sendiri.

- Ada batas waktu eksekusi dan batas jumlah baris yang dikembalikan.

4. Struktur Folder

```
rag-backend/  
  app/  
    main.py          titik masuk aplikasi FastAPI  
    config.py        konfigurasi terpusat (model, API key, path)  
    api/  
      routes_documents.py  endpoint upload dokumen dan Excel/CSV  
      routes_chat.py       endpoint percakapan  
      routes_admin.py      endpoint pemeriksaan/kesehatan sistem  
    core/  
      llm.py              pemanggilan LLM lewat LiteLLM  
      embeddings.py       pemanggilan model embedding  
      chunking.py         pemecahan dokumen lewat Docling  
      excel_ingest.py      parsing Excel/CSV ke SQLite  
      sql_tool.py         eksekusi SQL yang dibatasi (read only)  
      vectorstore.py       penyimpanan dan pencarian vektor di TurboVec  
      rag_chain.py         orkestrasi alur RAG dan tool calling  
      vlm.py              analisis gambar/tabel lewat model vision  
    db/  
      metadata_store.py   pencatatan metadata dokumen dan chunk  
    models/  
      schemas.py          struktur data request/response API  
  requirements.txt  
  .env.example  
  
rag-frontend/  
  app/  
    page.tsx           halaman utama  
    layout.tsx          pengaturan tampilan dan font  
  components/  
    Sidebar.tsx         daftar dokumen dan tombol unggah  
    UploadDropzone.tsx  area unggah dokumen  
    ChatPanel.tsx       area percakapan  
    MessageBubble.tsx   tampilan pesan dan sitasi sumber  
  lib/  
    api.ts              kumpulan pemanggilan ke backend  
  package.json  
  .env.local.example
```

5. Tantangan Selama Pengembangan

Beberapa kendala teknis yang ditemui dan cara penyelesaiannya, untuk referensi jika terjadi lagi di lingkungan lain:

Konflik Python versi Microsoft Store. Python yang terpasang di Windows lewat Microsoft Store memakai sandbox yang menyebabkan library `torch` (dipakai untuk OCR dan deteksi tabel) gagal dimuat dengan error DLL. Solusinya adalah memasang Python versi resmi dari python.org (bukan lewat Microsoft Store maupun "Python Install Manager" bawaan situs tersebut, karena keduanya masih memakai mekanisme yang serupa).

Urutan import memengaruhi library `torch` di Windows. Docling memuat beberapa library C lain sebelum sempat memuat `torch` secara internal, dan urutan ini bisa memicu konflik DLL. Solusinya adalah memastikan `torch` diimpor lebih dulu secara eksplisit sebelum Docling.

Ketidakcocokan versi antar paket Docling. Versi `docling` yang dipakai sudah agak lama, sementara `docling-core` yang terpasang otomatis jauh lebih baru dan butuh beberapa dependensi tambahan (`tree-sitter`, `tree-sitter-languages`, ekstra `docling-core[chunking]`) yang tidak ikut terpasang otomatis. Ditemukan satu per satu lewat pesan error saat aplikasi dijalankan.

Dokumentasi TurboVec berbeda dari API sebenarnya. TurboVec adalah library yang relatif

baru, sehingga nama class dan parameter yang benar (`IdMapIndex`, `bit_width`, `add_with_ids`) baru diketahui setelah memeriksa langsung sumber kodenya, bukan dari asumsi awal.

Variable lingkungan tidak otomatis terbaca oleh LiteLLM. File `.env` dibaca ke dalam objek konfigurasi aplikasi sendiri, tetapi LiteLLM membaca API key langsung dari environment variable sistem. Solusinya adalah mengekspor nilai dari `.env` ke `os.environ` secara eksplisit saat aplikasi dimulai.

Celah keamanan pada Next.js. Versi Next.js yang dipakai awalnya (14.2.5) memiliki beberapa celah keamanan yang sudah diperbaiki di versi 14.2.35. Perlu diperbarui sebelum aplikasi frontend ini dipakai di luar lingkungan pengembangan lokal.

Unduhan model pertama kali cukup lama. Docling dan `sentence-transformers` mengunduh beberapa model machine learning (ratusan megabita) dari internet saat pertama kali dipakai. Proses ini hanya terjadi sekali karena hasilnya disimpan dalam cache lokal.

6. Cara Menjalankan Proyek

6.1 Menjalankan Backend

Persyaratan: Python 3.11 terpasang dari installer resmi python.org (bukan Microsoft Store).

```
cd rag-backend
python -m venv .env
.venv\Scripts\Activate.ps1      (Windows PowerShell)
pip install -r requirements.txt
copy .env.example .env
```

Buka file `.env`, isi minimal satu API key sesuai model default yang dipakai, contoh:

```
OPENROUTER_API_KEY=isi_api_key_di_sini
```

Jalankan servernya:

```
uvicorn app.main:app --reload --port 8000
```

Server berjalan di `http://localhost:8000`. Dokumentasi API interaktif (Swagger UI) tersedia di `http://localhost:8000/docs`.

6.2 Menjalankan Frontend

Persyaratan: Node.js terpasang, dan backend sudah berjalan lebih dulu di port 8000.

```
cd rag-frontend
npm install
copy .env.local.example .env.local
npm run dev
```

Buka `http://localhost:3000` di browser.

6.3 Catatan Penting

- Backend dan frontend harus dijalankan bersamaan di dua terminal terpisah agar frontend bisa terhubung ke backend.
- Model default yang dipakai adalah DeepSeek lewat OpenRouter (untuk percakapan) dan `all-MiniLM-L6-v2` lokal (untuk embedding). Keduanya bisa diganti lewat file `.env` tanpa mengubah kode.
- Sebelum dipakai di luar lingkungan pengembangan lokal, disarankan memperbarui Next.js ke versi 14.2.35 atau lebih baru untuk menutup celah keamanan yang disebutkan pada bagian

5.

7. Ringkasan Kemampuan Sistem Saat Ini

- Mengunggah dan mengindeks dokumen PDF, DOCX, dan PPTX, termasuk ekstraksi tabel.
- Mengunggah dan mengindeks file Excel dan CSV sebagai tabel yang bisa dihitung secara presisi lewat SQL.
- Menjawab pertanyaan berbasis dokumen dengan sitasi sumber yang bisa ditelusuri balik.
- Model LLM, model embedding, dan model VLM (analisis gambar) dapat diganti lewat konfigurasi, tanpa mengubah kode aplikasi.
- Koleksi vektor otomatis terpisah bila model embedding yang dipakai berbeda.